

Labwork 4 - CUDA Threads

Do Minh Quang - 2540095
University of Science and Technology of Hanoi

1 GPU 2D Block

This labwork focuses on converting an RGB image to grayscale on the GPU using 2D thread blocks and comparing its performance with 1D implementation. The experiment uses 3167×3167 RGB image.



Figure 1: RGB image.

The 2D implementation extends the 1D approach by organizing threads and blocks along two dimensions. Each thread processes one pixel, with its position determined by its coordinates within the block and grid. Unlike the 1D implementation, the 2D version directly processes the original image without flattening the input.

```
@cuda.jit
def rgb2gray_gpu_2d(rgb, gray):
    tidx = cuda.threadIdx.x + cuda.blockIdx.x * cuda.
        blockDim.x
    tidy = cuda.threadIdx.y + cuda.blockIdx.y * cuda.
        blockDim.y
    g = np.uint8((rgb[tidy, tidx, 0] + rgb[tidy, tidx,
        1] + rgb[tidy, tidx, 2]) / 3)
```

```
gray[tidy, tidy, 0] = gray[tidy, tidy, 1] = gray[
    tidy, tidy, 2] = g
```

2 Results

Both implementations are executed 10 times, and the average execution time and standard deviation are recorded. A warm-up execution is performed before measurement to reduce the impact of initial CUDA overhead. The comparison uses a 1D block size of 1024 threads and a 2D block size of (32, 32), which also contains 1024 threads per block. The 1D implementation takes an average of 0.0201 seconds, while the 2D implementation takes 0.0200 seconds. These results indicate nearly identical performance between the two configurations.

2.1 Impact of block size

The block size is varied and the corresponding execution times are recorded. Each configuration is executed 10 times, and the average execution time is reported. The results are presented in Figure 2.

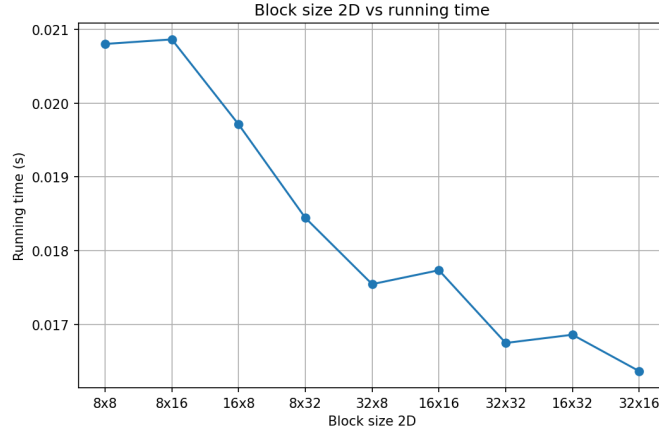


Figure 2: GPU execution time for different block sizes.

The results show that block size has a noticeable impact on execution time. Small configurations such as (8, 8) and (8, 16) show the highest execution times, around 0.021 seconds. As the block dimensions increase, the execution time generally decreases. The best performance is obtained with

the (32, 16) configuration, achieving an average execution time of 0.0164 seconds.

3 Answering questions

3.1 Question 1

The GPU has a maximum capacity of 512 threads per block and 1024 threads per streaming multiprocessor (SM). Among the given configurations, (32, 32) is not valid because it requires 1024 threads/block which exceeds the hardware limit. For the (8, 8) configuration, each block contains 64 threads. Therefore, one SM could theoretically accommodate 16 blocks. However, the GPU supports a maximum of 8 blocks per SM, so the block limit becomes the restricting factor.

Therefore, the selected configuration is (16, 16), with each block contains 256 threads and 4 blocks/SM.

3.2 Question 2

With the 128 and 256 threads/block configurations achieve the maximum of 512 and 1024 threads if using all 4 blocks. The 1024 threads/block configuration cannot fully utilize the SM capacity because the block limit restricts execution to a single block, leaving 512 unused threads.

Therefore, using 3 blocks of 512 threads can fully utilize all 1536 threads and still ensure the number of block not exceeds 4.

3.3 Question 3

We need at least 4 blocks of 512 threads to process all 2000 elements in the vector. So it will produce a grid of $512 \times 4 = 2048$ threads.